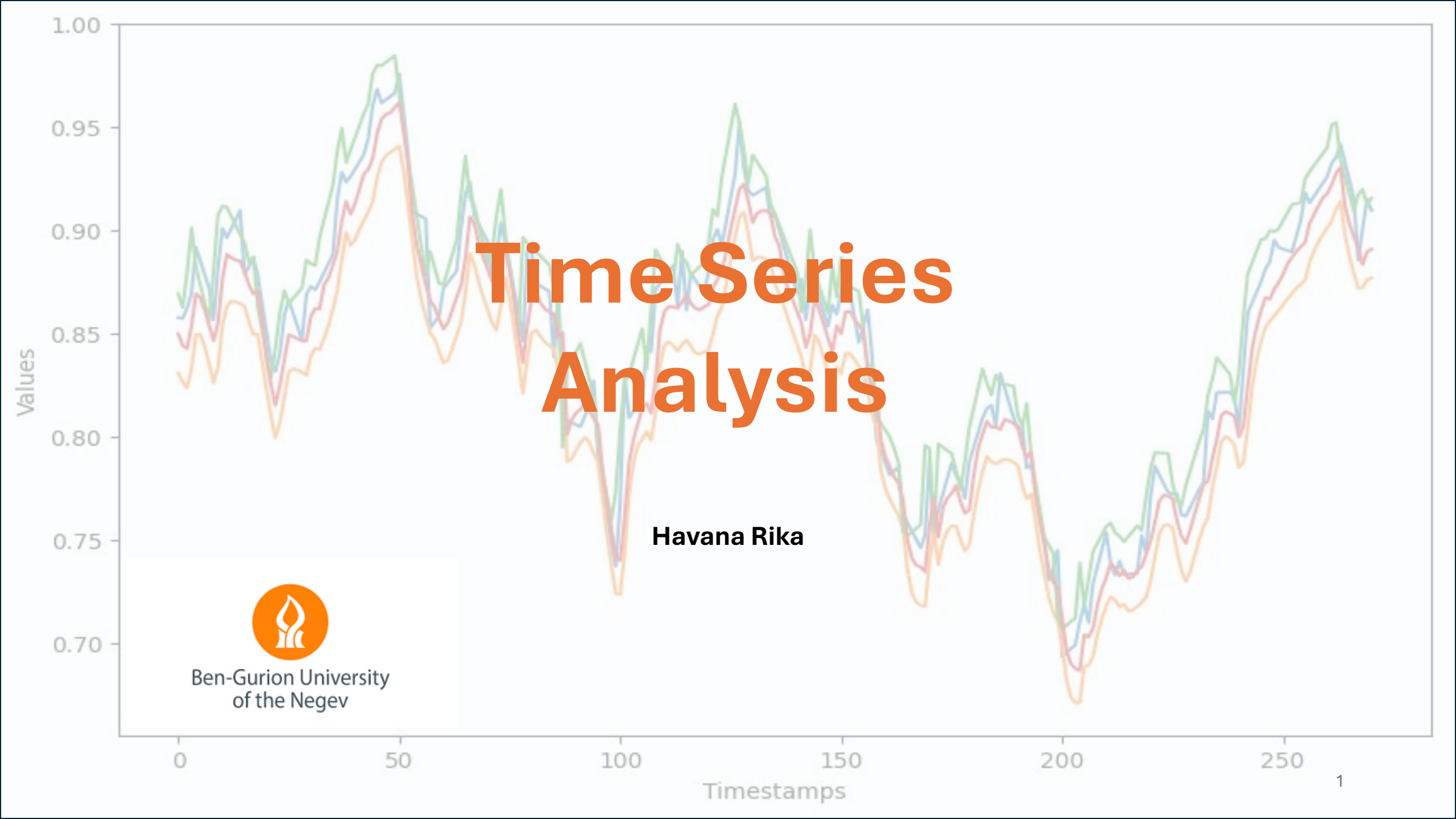


Time Series Analysis

Havana Rika

Ben-Gurion University of the Negev

Preprocessing Time-Series

Preprocessing is a crucial step before applying machine learning to time-series data.

Real-world data is often messy, incomplete, inconsistent, noisy, and collected from multiple sources.

Good preprocessing improves data quality and can strongly affect model performance.

Why Preprocessing Matters

Machine learning models are only as good as the data they receive.

Low-quality input can lead to unreliable or invalid results:

“garbage in, garbage out.”

Preprocessing often takes most of the work in a real ML project but can produce large performance gains.

Main Preprocessing Techniques

- Feature transforms
  - Scaling
  - Log and power transformations
  - Imputation
- Feature engineering

Feature Transforms

Feature transforms **modify raw features** so they better match model assumptions.

Many classical **models work better** when **features** are approximately **normally distributed**.

Transforms can **reduce skewness**, **stabilize variance**, and make features **easier** to **compare**.

Scaling

Scaling adjusts the range or distribution of numeric features.

It is especially important for models sensitive to feature magnitude, such as linear models and distance-based methods.

Min-Max Scaling

Min-max scaling maps values into a fixed range, usually between 0 and 1 (if $a = 0, b = 1$).

It **preserves** the **relative order** of values but changes their scale.

Useful when features have different natural ranges.

$$x' = b\frac{a + x - \min(x)}{\max(x) - \min(x)}$$

Z-Score Normalization

Z-score normalization transforms a feature to have **mean 0** and **standard deviation 1**.

It is useful when a model assumes **centered** and **comparable features**.

If the original data is **Gaussian**, the transformed data becomes **standard normal**.

$$x' = \frac{x - \overline{x}}{\sigma}$$

Log Transformation

Log transformation compresses large values and **reduces skewed distributions**.

The range between 0 and 1 gets mapped to negative numbers $(-\infty, 0)$, while numbers $x>=1$ get compressed in the positive range.

It is useful when the feature follows a log-normal distribution.

Always inspect the data before and after applying the transformation.

Power Transformations

Power transformations try to make data more normal-like while preserving order (monotonicity).

$$f(x) = cx^n$$

They can **improve** the **validity** of classical statistical and **forecasting models**.

This reduces the transformation to the optimal choice of the parameter λ .

For practical purposes, two power transformations are most commonly used:

- Box-Cox transformation
- Yeo–Johnson

A power transformation is generally defined like this:

$$x_i^{(\lambda)} = \begin{cases} \dfrac{x_i^{\lambda} - 1}{\lambda(\mathrm{GM}(x))^{\lambda-1}} & \text{if } \lambda \neq 0 \\ \mathrm{GM}(x) \ln y_i & \text{if } \lambda = 0 \end{cases}$$

where $GM(x)$ is the geometric mean of x:

$$\mathrm{GM}\,(x) = \left(\prod_{i=1}^{n} x_i\right)^{\frac{1}{n}} = \sqrt[n]{x_1 x_2 \cdots x_n}$$

Box-Cox Transformation

Box-Cox applies a power transformation controlled by a parameter lambda.

When lambda equals 0, Box-Cox is equivalent to a log transformation.

It works only with positive values.

$$x_i^{(\lambda)} = \begin{cases} \dfrac{x_i^{\lambda} - 1}{\lambda} & \text{if } \lambda \neq 0, \\ \ln\ x_i & \text{if } \lambda = 0, \end{cases}$$

Yeo-Johnson Transformation

Yeo-Johnson extends Box-Cox to **support zero** and **negative values**.

It is useful when the data contains values that cannot be handled by the standard Box-Cox.

$$x_i^{(\lambda)} = \begin{cases} \dfrac{(x_i+1)^{\lambda}-1}{\lambda} & \text{if } \lambda \neq 0, x \geq 0 \\ \log(x_i+1) & \text{if } \lambda = 0, x \geq 0 \\ -\left[\dfrac{(-x_i+1)^{(2-\lambda)}-1}{2-\lambda}\right] & \text{if } \lambda \neq 2, x < 0 \\ -\log(-x_i+1) & \text{if } \lambda = 2, x < 0 \end{cases}$$

Power Transformations Advantages

These two transformations (Box-Cox and Yeo-Johnson) are the most popular for time series analysis because they address one of the most common and challenging issues in modeling and forecasting: **variance instability over time (heteroscedasticity) and deviation from normality**.

While trend and seasonality are typically handled using differencing, power transformations address the underlying dispersion structure of the data itself.Here are the key reasons why these two are the industry standard:

1. **Variance Stabilization**

    - Both transformations mathematically **"compress" higher** values and **stretch lower values** in an optimal way. This stabilizes the variance over time, which dramatically improves model accuracy and the reliability of forecast confidence intervals.

2. **Generalization of Standard Transformations (Flexible λ Parameter)**

    - The algorithm **automatically finds the optimal λ**(usually via **Maximum Likelihood Estimation**) that best fits the unique structure of your data, eliminating the need for manual trial and error.

Quantile Transformation

Quantile transformation **maps** a **feature** according to its **cumulative distribution**.

It maps a feature to the **uniform** distribution based on an estimate of the cumulative distribution function.

Optionally, this can then be mapped in a second step to a **normal** distribution.

The advantage of this transformation is that it makes features more **convenient** to **process** and plot, and **easier** to **compare**, even if they were measured at different scales.

Log Transformations in Practice

```
from scipy.optimize import minimize
import numpy as np
np.random.seed(0)
```

```
pts = 10000
vals = np.random.lognormal(0, 1.0, pts)
```

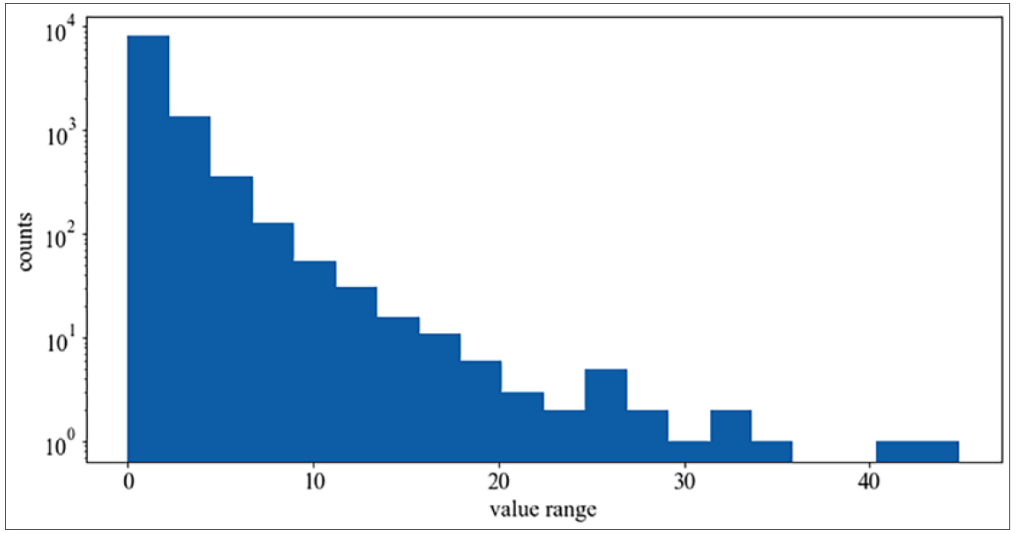
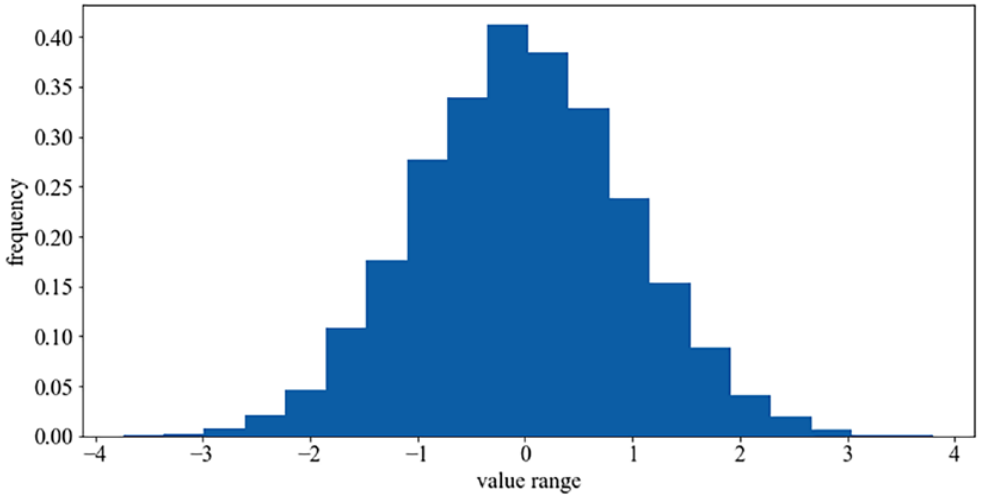


Figure 3.2: An array sampled from a lognormal distribution

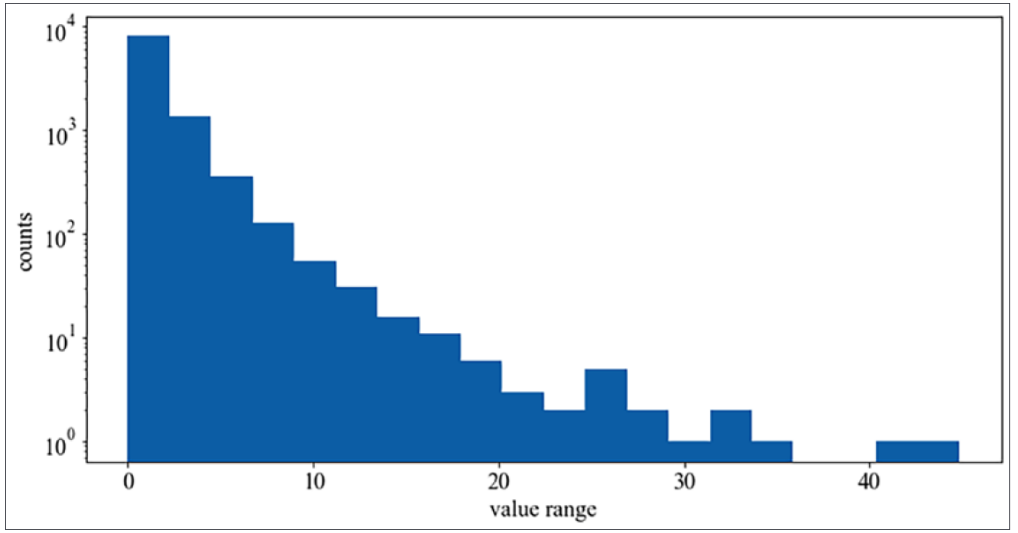
```
log_transformed = np.log(vals)
_, p = normaltest(log_transformed)
print(f"significance: {p:.2f}")
```



BoxCox Transformations in Practice

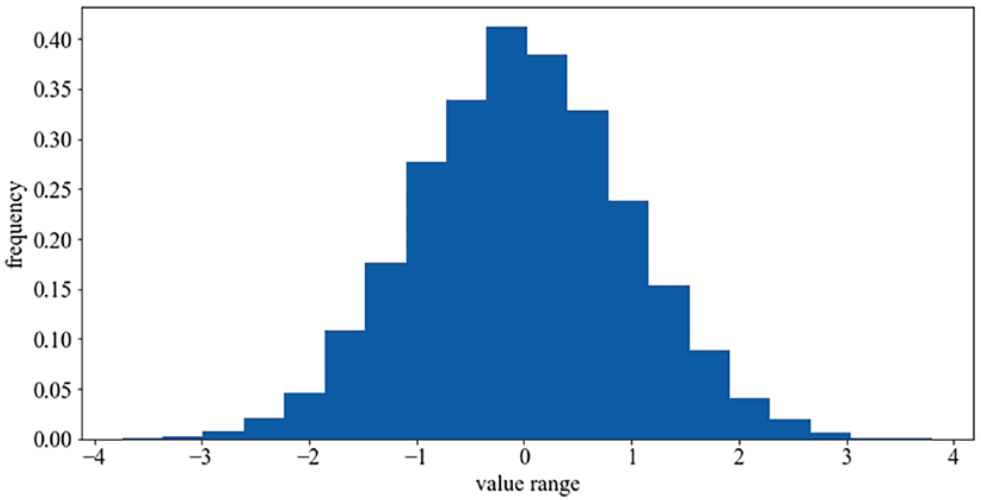
```
from scipy.optimize import minimize
import numpy as np
np.random.seed(0)
```

```
pts = 10000
vals = np.random.lognormal(0, 1.0, pts)
```



We can also apply Box-Cox transformation:

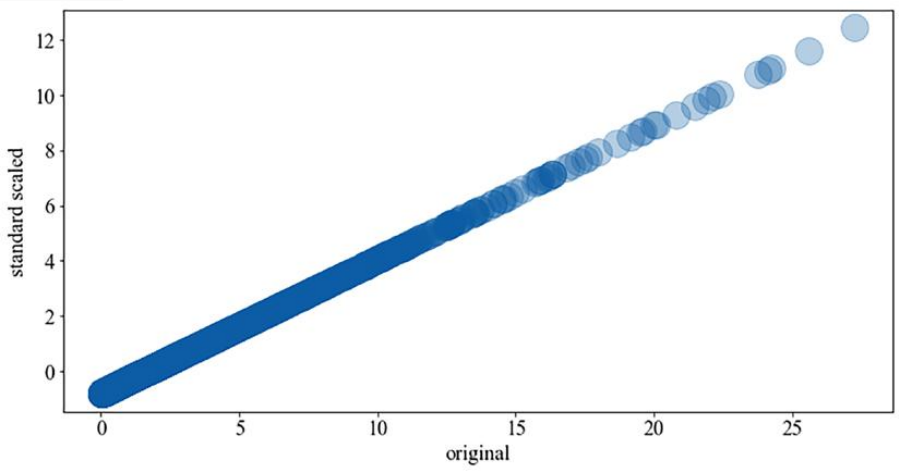
```
from scipy.stats import boxcox
vals_bc = boxcox(vals, 0.0)
```



Standard Normalization & Min Max

```
from sklearn.preprocessing import StandardScaler
from scipy.stats import normaltest
scaler = StandardScaler()
vals_ss = scaler.fit_transform(vals.reshape(-1, 1))
_, p = normaltest(vals_ss)
print(f"significance: {p:.2f}")
```

```
from sklearn.preprocessing import minmax_scale
vals_mm = minmax_scale(vals)
_, p = normaltest(vals_mm.squeeze())
print(f"significance: {p:.2f}")
```



Normality Testing

Standard scaling and **min-max** scaling change the scale but **do not change the shape** of the distribution.

Log transformation can make lognormal data approximately normal.

Box-Cox with lambda 0 gives the same result as the log transformation.

Imputation

Imputation replaces missing values.

This is necessary because many machine learning algorithms cannot handle missing values directly.

Types of Imputation:

Unit imputation replaces missing values with constants such as 0, mean, or median.

Model-based imputation predicts missing values using other variables.

Unit imputation is simpler and widely used.

Python Practice: Imputation

```
import numpy as np
from sklearn.impute import SimpleImputer
imp_mean = SimpleImputer(missing_values=np.nan, strategy='mean')
```

```
imp_mean.fit([[7, 2, 3], [4, np.nan, 6], [10, 5, 9]])
SimpleImputer()
df = [[np.nan, 2, 3], [4, np.nan, 6], [10, np.nan, 9]]
print(imp_mean.transform(df))
```

We get the following imputed values:

```
[[ 7.   2.   3. ]
 [ 4.   3.5  6. ]
 [10.   3.5  9. ]]
```

Handling missing data

When working with large datasets, **missing values** should not be handled automatically by dropping rows or filling with the mean.

First, consider the **data-generating process**. Sometimes what looks like missing data actually carries meaning. For example, if a supermarket product has no transactions on a day, that usually means **zero sales**, not missing data, so those gaps should be filled with zero.

If the missingness follows a **pattern**, such as data being absent every Sunday, handling it depends on the model you plan to use. Filling those gaps with zeros can mislead some forecasting models unless you also include information like the day of the week so the model can recognize the pattern.

Finally, if a product that normally sells well suddenly shows **zero sales**, that may indicate a real data problem or special event, such as a point-of-sale (POS) failure, entry error, or out-of-stock situation. In those cases, the values may need to be **imputed carefully** using appropriate techniques.

The Air Quality dataset

The Air Quality dataset **published by the ACT Government**, Canberra, Australia, under the CC by Attribution 4.0 International License (https://www.data.act.gov.au/Environment/Air-Quality-Monitoring-Data/94a5-zqnn) and see how we can **impute** such **values** using **pandas**.

```
df = pd.read_csv("au_airquality.csv", sep=",")
df.head()
```

	name	gps	datetime	no2	o3_1hr	o3_4hr	o3_8hr	co	pm10_1_hr	pm2_5_1_hr	...	aqi_cc
0	Monash	\n, \n(-35.418302, 149.094018)	2024-03-18T16:00:00.000	NaN	NaN	NaN	0.019	NaN	8.5	2.1	...	NaN
1	Monash	\n, \n(-35.418302, 149.094018)	2024-02-22T16:00:00.000	NaN	NaN	NaN	0.024	NaN	0.0	0.0	...	NaN
2	Monash	\n, \n(-35.418302, 149.094018)	2024-02-15T16:00:00.000	NaN	NaN	NaN	0.016	NaN	9.4	NaN	...	NaN
3	Monash	\n, \n(-35.418302, 149.094018)	2024-02-10T16:00:00.000	NaN	NaN	NaN	0.018	NaN	5.4	4.1	...	NaN
4	Florey	\n, \n(-35.220606, 149.043539)	2024-01-15T16:00:00.000	NaN	NaN	NaN	0.017	NaN	2.6	1.4	...	NaN

5 rows × 22 columns

The Air Quality dataset

```
#Selecting one location and pm2.5
df = df.loc[df.name=="Monash", ['datetime', 'pm2_5_1_hr']]
df.dtypes
```

	0
datetime	object
pm2_5_1_hr	float64

dtype: object

```
df.datetime = pd.to_datetime(df.datetime)
df.sort_values("datetime", inplace=True)
df.set_index("datetime", inplace=True)
df.head()
```

	pm2_5_1_hr
datetime	
2012-08-31 16:00:00	NaN
2013-03-18 16:00:00	NaN
2014-02-24 21:00:00	NaN
2014-04-24 16:00:00	NaN
2015-01-28 05:00:00	NaN

Artificial missing values

We have chosen the region **Monash** and **PM2.5** readings, and artificially introduced some missing values, as shown in the following diagram

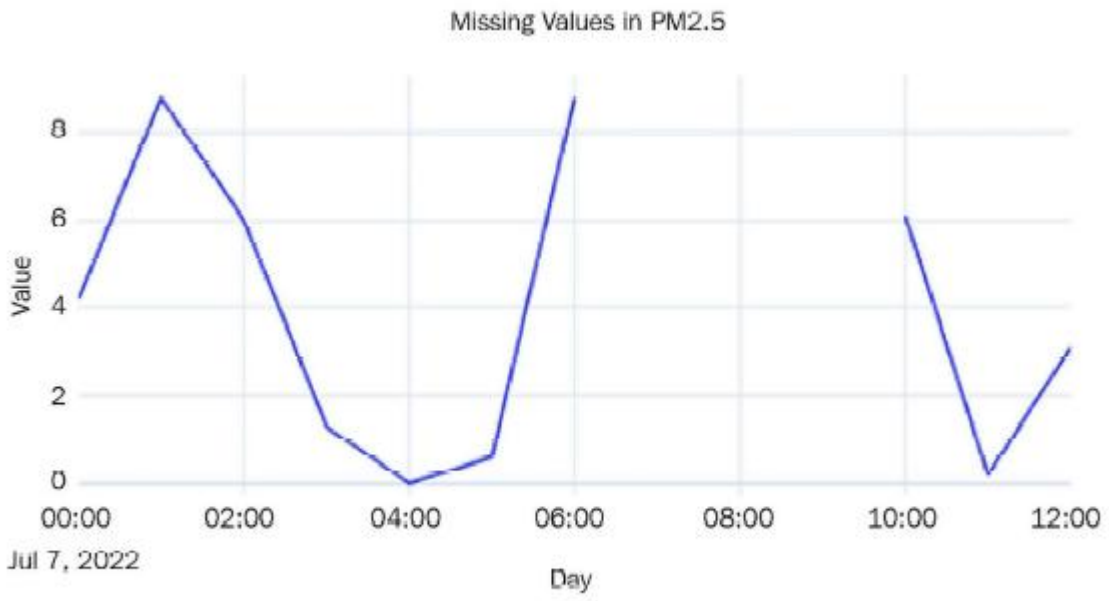


Figure 2.3: Missing values in the Air Quality dataset

Techniques to fill the missing values

- **Last Observation Carried Forward or Forward Fill:** This imputation technique takes the last observed value, using that to fill in all the missing values until it finds the next observation. This is also called forward fill.

- **Next Observation Carried Backward of Backward Fill:** This imputation technique takes the next observation and backtracks to fill in all the missing values with this value. This is also called backward fill.

- **Mean Value Fill:** This imputation technique is also pretty simple. We calculate the mean of the entire series, and wherever we find missing values, we fill it with the mean value.

Forward, Backward, and Mean Value Fill

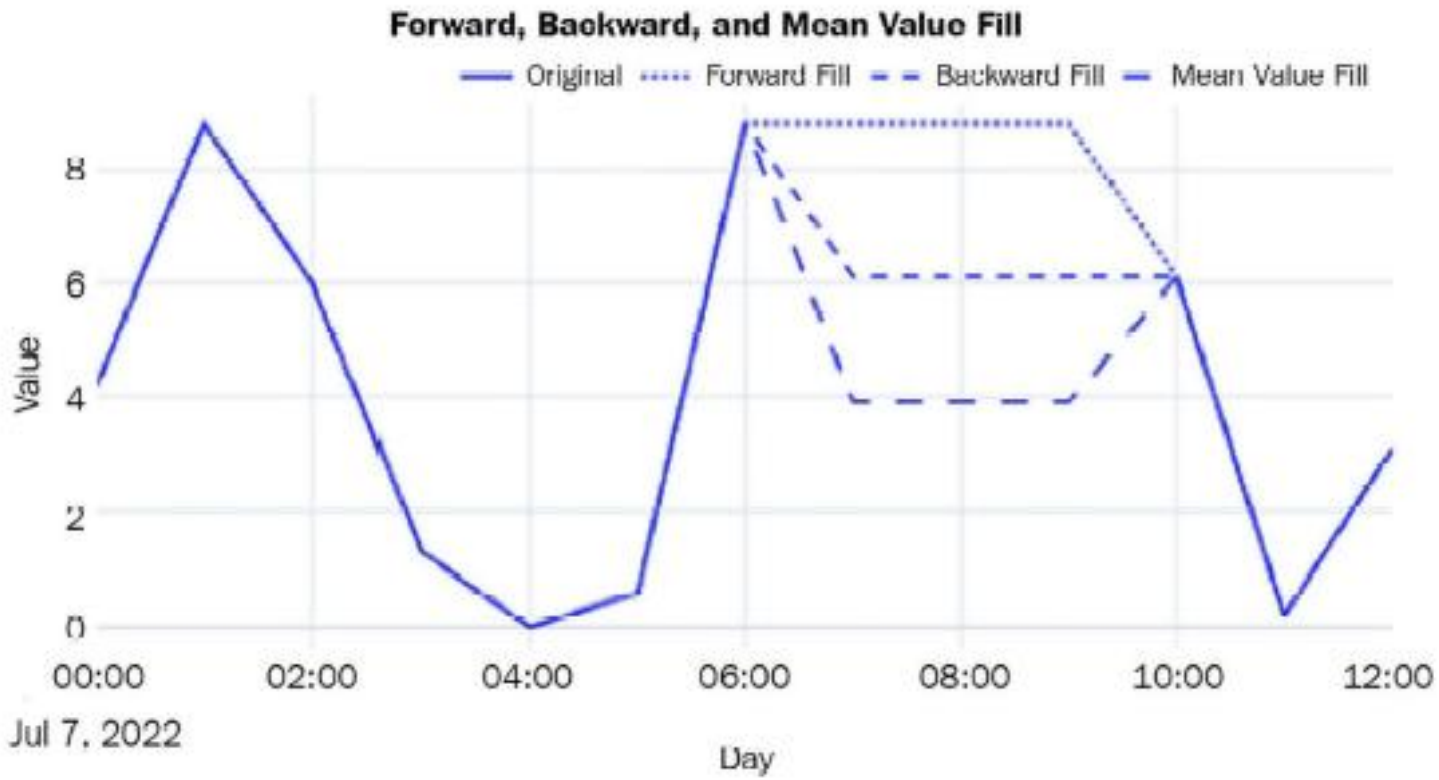


Figure 2.4: Imputed missing values using forward, backward, and mean value fill

Linear and Nearest Interpolation

- **Linear Interpolation**: Linear interpolation is just like drawing a line between the two observed points and filling in the missing values so that they lie on this line.

- **Nearest Interpolation:** This is intuitively like a combination of the forward and backward fill. For each missing value, the closest observed value is found and used to fill it in.

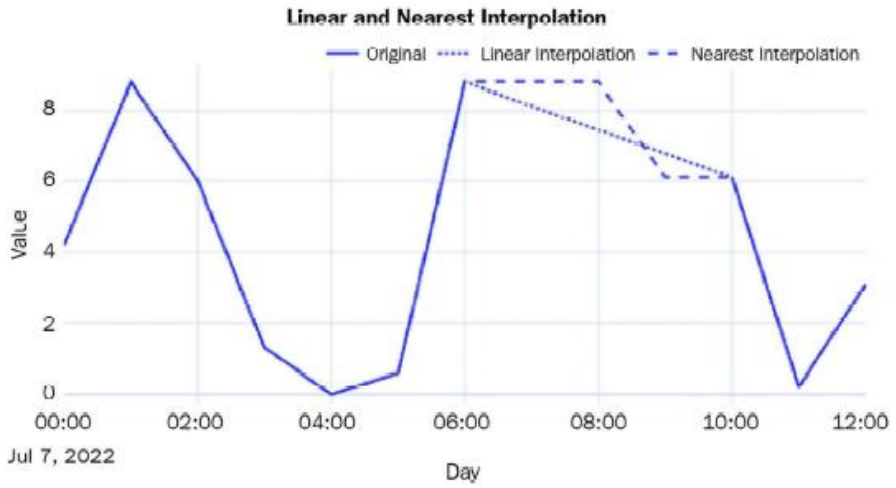


Figure 2.5: Imputed missing values using linear and nearest interpolation

Non-linear Interpolation

- **Spline, Polynomial, and Other Interpolations:** In addition to linear interpolation, pandas also supports non-linear interpolation techniques that call a SciPy routine at the backend.

- **Spline** and **polynomial** interpolations are **similar**. They fit a spline/polynomial of a given order to the data and use that to fill in missing values. While using spline or polynomial as the method in interpolate, **we should always provide order** as well.

- The higher the order, the more flexible the function that is used to fit the observed points will be.

```
_df = df.copy()
_df["spline_interpolation"] = _df['pm2_5_1_hr'].interpolate(method="spline", order=2)
_df["polynomial_interpolation"] = _df['pm2_5_1_hr'].interpolate(method="polynomial", order=5)
```

Spline and Polynomial Interpolation

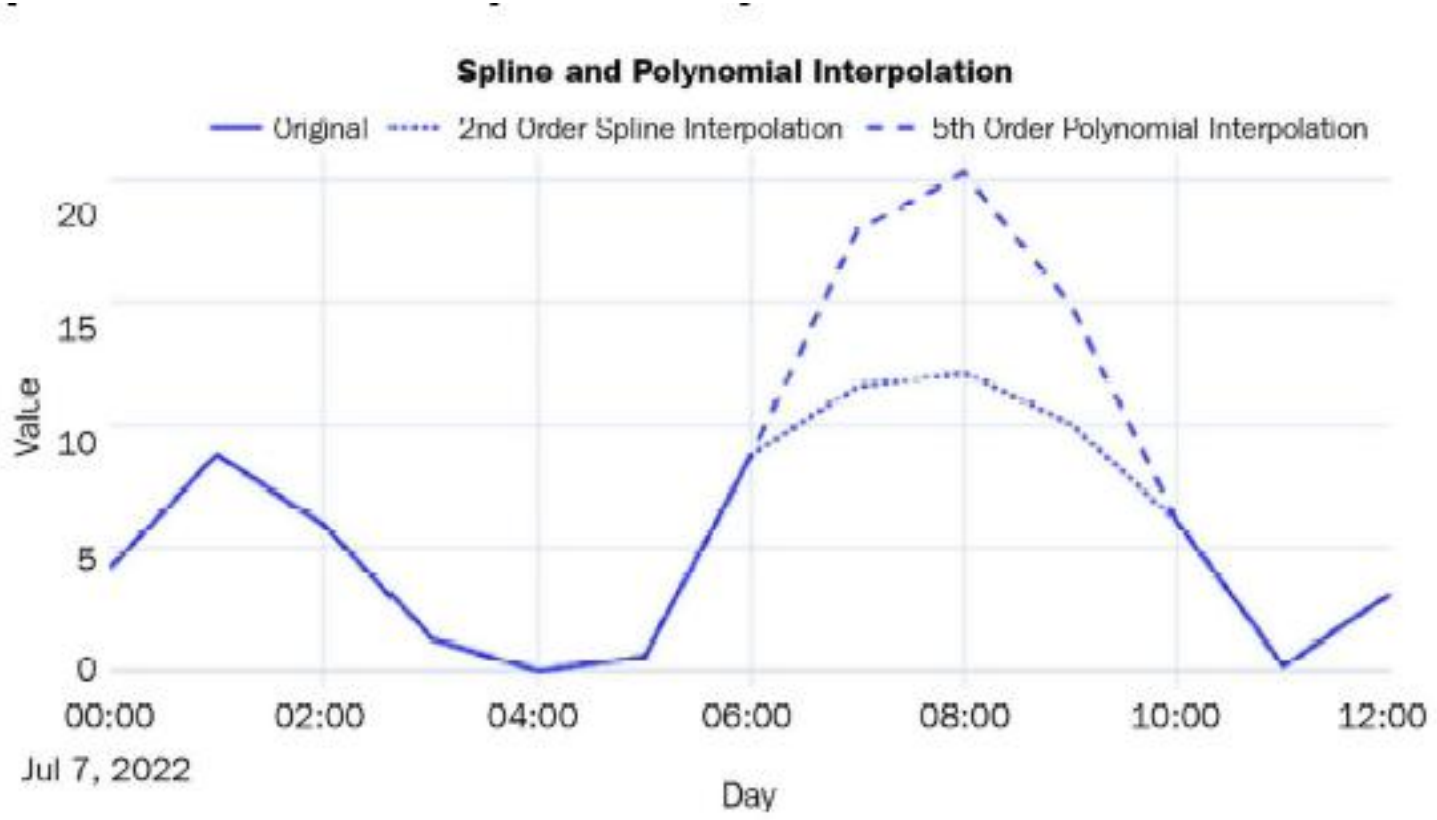


Figure 2.6: Imputed missing values using spline and polynomial interpolation

Smart meters in London

The dataset contains **energy consumption** readings for a sample of **5,567 London households** that took part in the UK Power Networks-led Low Carbon London project between November 2011 and February 2014. Readings were taken at **half-hourly intervals**.

The available metadata:

- CACI UK segmented the UK's population into **demographic types**, called Acorn.

- The dataset contains two groups of customers. One group who was subjected to **dynamic timeof-use (dToU)** energy prices throughout 2013, and another group who were on **flat-rate tariffs**. The tariff prices for the dToU were given **a day ahead**, via the smart meter IHD or text message.

- Jean-Michel D also enriched the dataset with **weather** and **UK bank holiday** data.

https://www.kaggle.com/datasets/jeanmidev/smart-meters-in-london

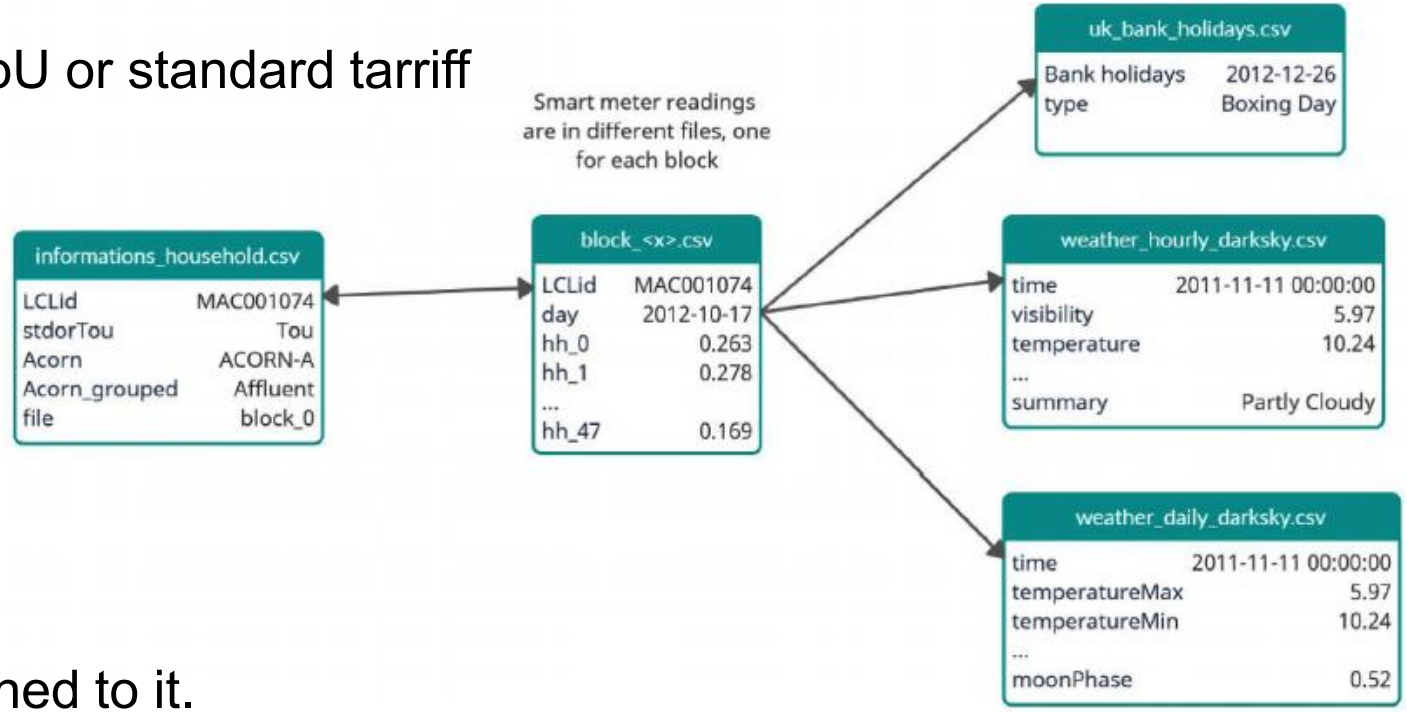
The Acorn Classes

Acorn Group	Acorn Class
Affluent Achievers	A-Lavish Lifestyles
	B-Executive Wealth
	C-Mature Money
Rising Prosperity	D-City Sophisticates
	E-Career Climbers
Comfortable Communities	F-Countryside Communities
	G-Successful Suburbs
	H-Steady Neighborhoods
	I-Comfortable Seniors
	J-Starting Out
Financially Stretched	K-Student Life
	L-Modest Means
	M-Striving Families
	N-Poorer Pensioners
Urban Adversity	O-Young Hardship
	P-Struggling Estates
	Q-Difficult Circumstances

Data Model of the London Smart Meters Dataset

Let's look at a few key column names and their meanings:

- **LCLid:** The unique consumer ID for a household
- **stdorTou:** Whether the household has dToU or standard tarriff
- **Acorn:** The ACORN class
- **Acorn_grouped:** The ACORN group
- **file:** The block number



Each **LCLid** has a **unique** time series attached to it.
The time series file is formatted in a slightly tricky format - **each day, there will be 48 observations at a half-hourly frequency** in the columns of the file.

Converting the half-hourly into TS data

A few general categories of information we will find in a time series dataset:

• **Time Series Identifiers:** These are identifiers for a particular time series. It can be a name, an ID, or any other unique feature - for example, the SKU name or the ID of a retail sales dataset or the consumer ID in the energy dataset that we are working with are all time series identifiers.

• **Metadata or Static Features:** This information does not vary with time. An example of this is the ACORN classification of the household in our dataset.

• **Time-Varying Features:** This information varies with time - for example, the weather information.

For each point in time, we have a different value for weather, unlike the Acorn classification.

Compact, expanded, and wide forms of data

Time series datasets can be organized in several formats. The traditional one is **wide format**, where the date acts like an index and **each separate time series gets its own column**.

This becomes **impractical** when there are many series, because the table grows very wide and cannot easily store metadata about each series.

This data format **does not allow us to include any metadata** about the time series.

From an operational perspective, the wide format also **does not play well with relational databases** because we have to keep adding columns to a table when we get new time series.

We won't be using this format in this book.

Compact, expanded, and wide forms of data

In **compact format**, each entire time series is stored in a single row, with metadata in normal columns and the time-varying values stored as arrays. **Extra columns like start_datetime and frequency are used to reconstruct the timeline**. This format is memory-efficient and works well for regularly sampled data.

LCLid	start_datetime	frequency	energy_consumption	series_length
MAC000002	2012-10-13	30min	[0.263, 0.2689999999999999, 0.275, 0.256, 0.21...	24144
MAC000246	2011-12-04	30min	[0.175, 0.098, 0.144, 0.065, 0.071, 0.037, 0.0...	39216
MAC000450	2012-03-23	30min	[0.337, 1.426, 0.996, 0.971, 0.994, 0.952, 0.8...	33936
MAC001074	2012-05-09	30min	[0.18, 0.086, 0.106, 0.173, 0.146, 0.223, 0.21...	31680
MAC003223	2012-09-18	30min	[0.076, 0.079, 0.123, 0.109, 0.051, 0.069, 0.0...	25344

Compact, expanded, and wide forms of data

In **expanded format** (also called **long format**), each time step of a series gets its own row. The identifiers and metadata are repeated across rows, and there is an explicit timestamp column. This format **is more flexible** and **works better for handling metadata and database operations**.

LCLid	energy_consumption	series_length	timestamp	frequency
MAC000002	0.263	24144	2012-10-13 00:00:00	30min
MAC000002	0.269	24144	2012-10-13 00:30:00	30min
MAC000002	0.275	24144	2012-10-13 01:00:00	30min
MAC000002	0.256	24144	2012-10-13 01:30:00	30min
MAC000002	0.211	24144	2012-10-13 02:00:00	30min

Enforcing regular intervals in time series

One of the first things you should check and correct is whether the regularly sampled time series data that you have has **equal intervals of time**.

In practice, even regularly sampled time series have some
samples missing in between, due to some data collection error or some other peculiar way data is collected.

So while working with the data, we will make sure we enforce regular intervals in the time series.

In our smart meters dataset, some **LCLid** columns end much earlier than the rest. Maybe the household opted out of the program, or they moved out and left the house empty; the reason could be anything.

However, we need to handle that while we enforce regular intervals.

Convert the London Smart Meters dataset to TS

For each dataset that you come across, the steps you would have to take to convert it into either a compact or expanded form would be different. It depends on how the original data is structured.

We will look at how the London Smart Meters dataset can be transformed.

There are two steps we need to do before we can start processing the data into either compact or expanded form:

1. **Find the Global End Date:** We must find the maximum date across all the block files so that we know the global end date of the time series.

2. **Basic Preprocessing:** If you remember how hhblock_dataset is structured, you will remember that each row has a date and that, along the columns, we have half-hourly blocks. We need to reshape that into a long form, where each row has a date and a single half-hourly block. It's easier to handle that way.

Handling longer periods of missing data

One of the best ways to visualize the missing data in a group of related time series is by using a very helpful package called `missingno`:

```
# Pivot the data to set the index as the datetime and the different time series
along the columns
plot_df = pd.pivot_table(exp_block_df, index="timestamp", columns="LCLid",
values="energy_consumption")
# Generate Plot. Since we have a datetime index, we can mention the frequency
to decide what do we want on the X axis
msno.matrix(plot_df, freq="M")
```

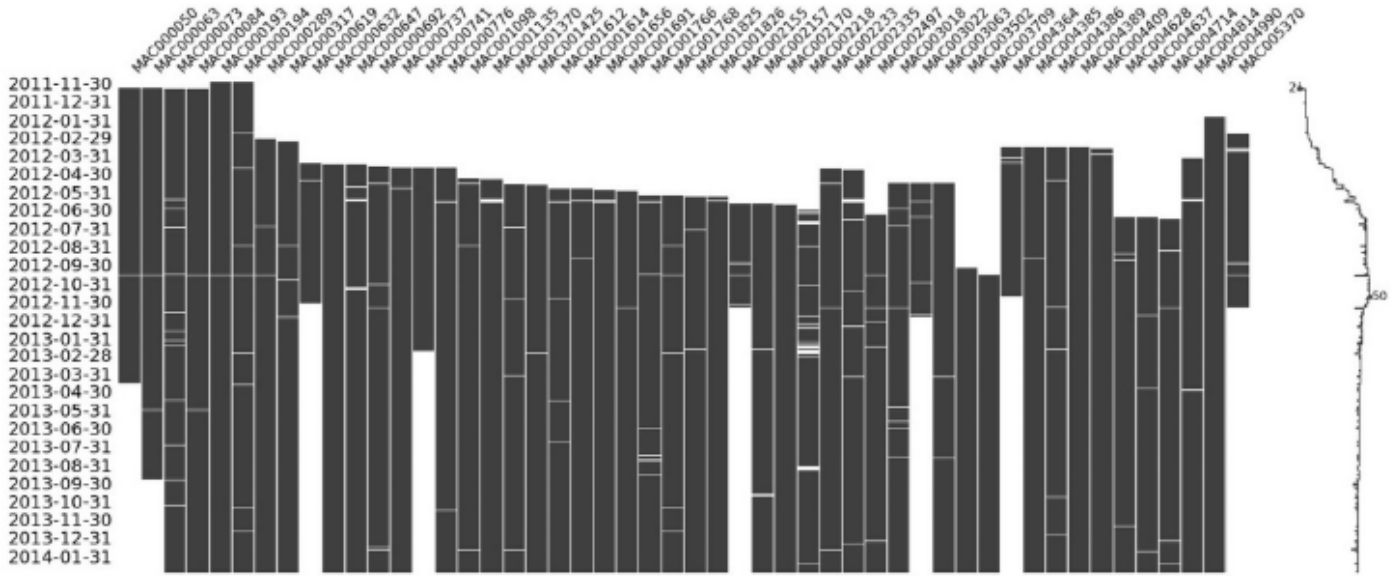


Figure 2.9: Visualization of the missing data in block 7

Handling longer periods of missing data

So for now, let's pick one `LCLid` and dig deeper. We already know that there are some missing values between `2012-09-30` and `2012-10-31`. Let's visualize that period:

```
# Taking a single time series from the block
ts_df = exp_block_df[exp_block_df.LCLid=="MAC000193"].set_index("timestamp")
msno.matrix(ts_df["2012-09-30": "2012-10-31"], freq="D")
```

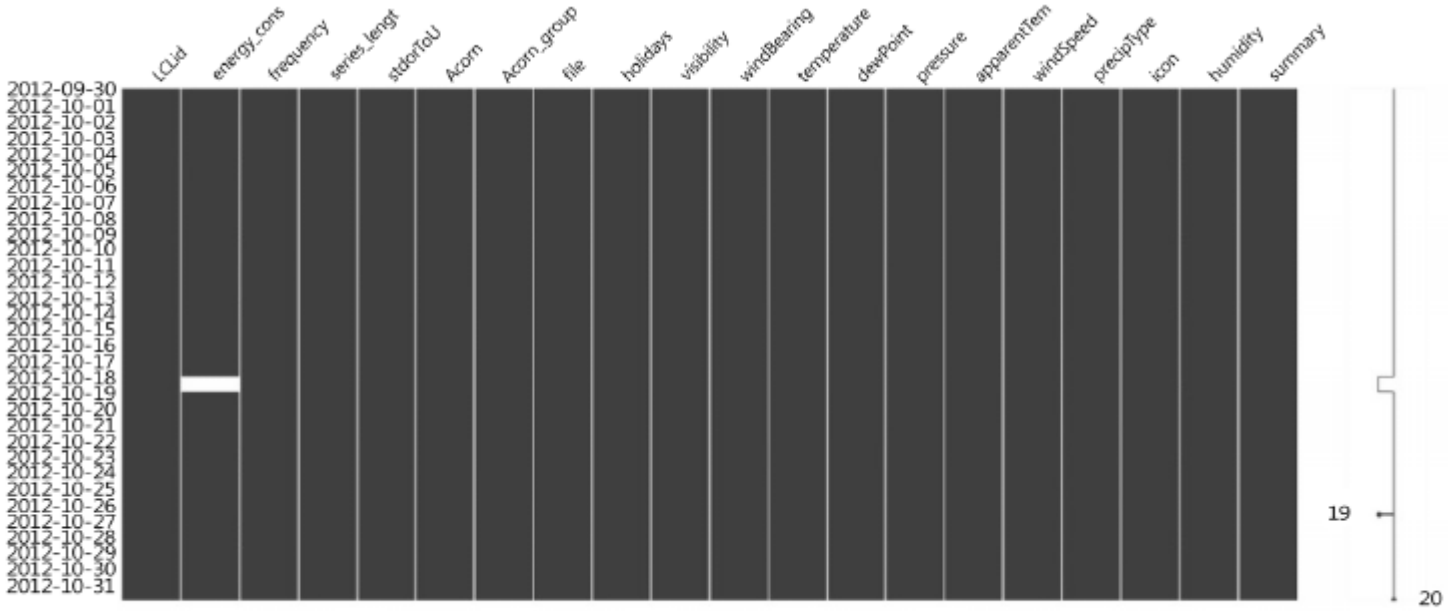


Figure 2.10: Visualization of missing data of MAC000193 between 2012-09-30 and 2012-10-31

Missing 2 days of energy consumption

```
# The dates between which we are nulling out the time series
window = slice("2012-10-07", "2012-10-08")
# Creating a new column and artificially creating missing values
ts_df['energy_consumption_missing'] = ts_df.energy_consumption
ts_df.loc[window, "energy_consumption_missing"] = np.nan
```

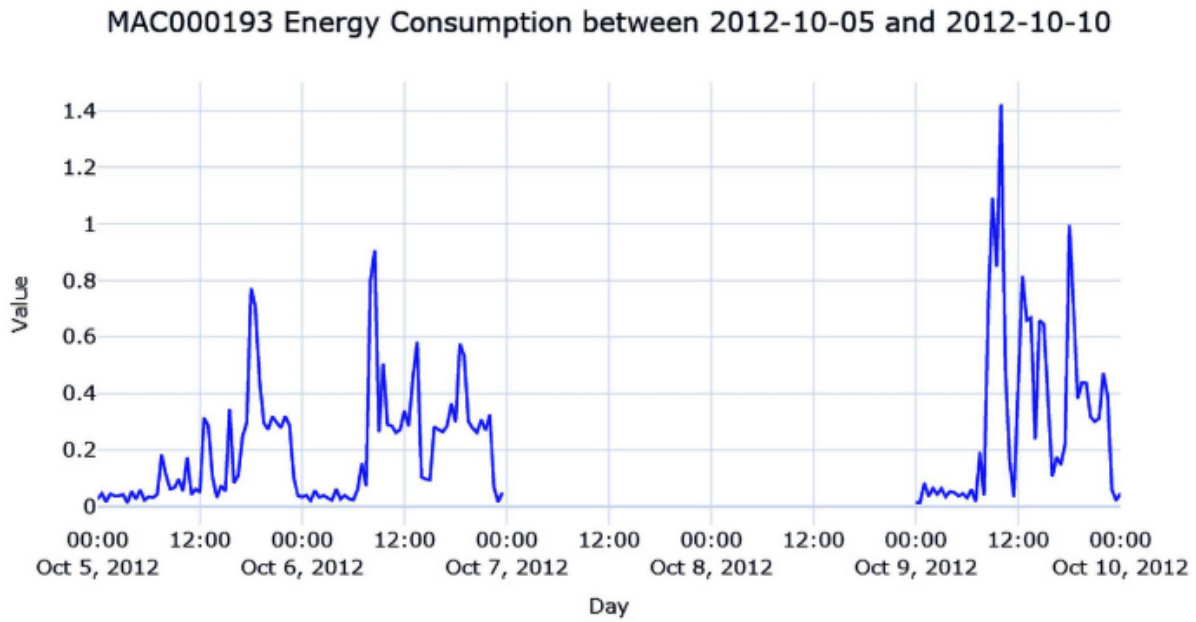


Figure 2.11: The energy consumption of MAC000193 between 2012-10-05 and 2012-10-10

Imputing with the previous day

```
#Shifting 48 steps to get previous day
ts_df["prev_day"] = ts_df['energy_consumption'].shift(48)
#Using the shifted column to fill missing
ts_df['prev_day_imputed'] =  ts_df['energy_consumption_missing']
ts_df.loc[null_mask,"prev_day_imputed"] = ts_df.loc[null_mask,"prev_day"]
mae = mean_absolute_error(ts_df.loc[window, "prev_day_imputed"], ts_
df.loc[window, "energy_consumption"])
```

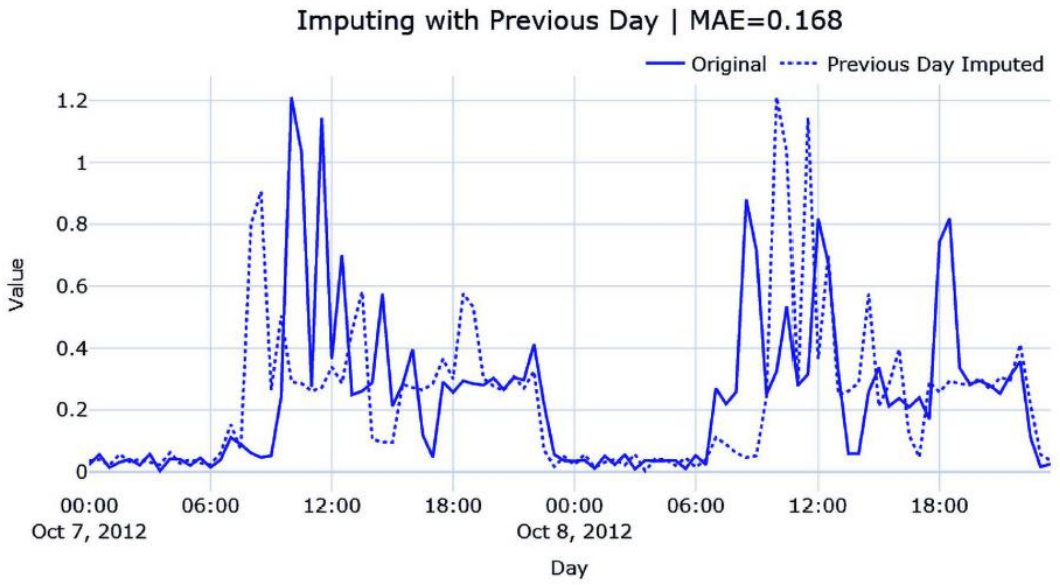


Figure 2.12: Imputing with the previous day

Hourly average profile

```
#Create a column with the Hour from timestamp
ts_df["hour"] = ts_df.index.hour
#Calculate hourly average consumption
hourly_profile = ts_df.groupby(['hour'])['energy_consumption'].mean().reset_
index()
hourly_profile.rename(columns={"energy_consumption": "hourly_profile"},
inplace=True)
#Saving the index because it gets lost in merge
idx = ts_df.index
#Merge the hourly profile dataframe to ts dataframe
ts_df = ts_df.merge(hourly_profile, on=['hour'], how='left', validate="many_to
one")
ts_df.index = idx
#Using the hourly profile to fill missing
```

```
ts_df['hourly_profile_imputed'] = ts_df['energy_consumption_missing']
ts_df.loc[null_mask,"hourly_profile_imputed"] = ts_df.loc[null_mask,"hourly_
profile"]
mae = mean_absolute_error(ts_df.loc[window, "hourly_profile_imputed"], ts_
df.loc[window, "energy_consumption"])
```

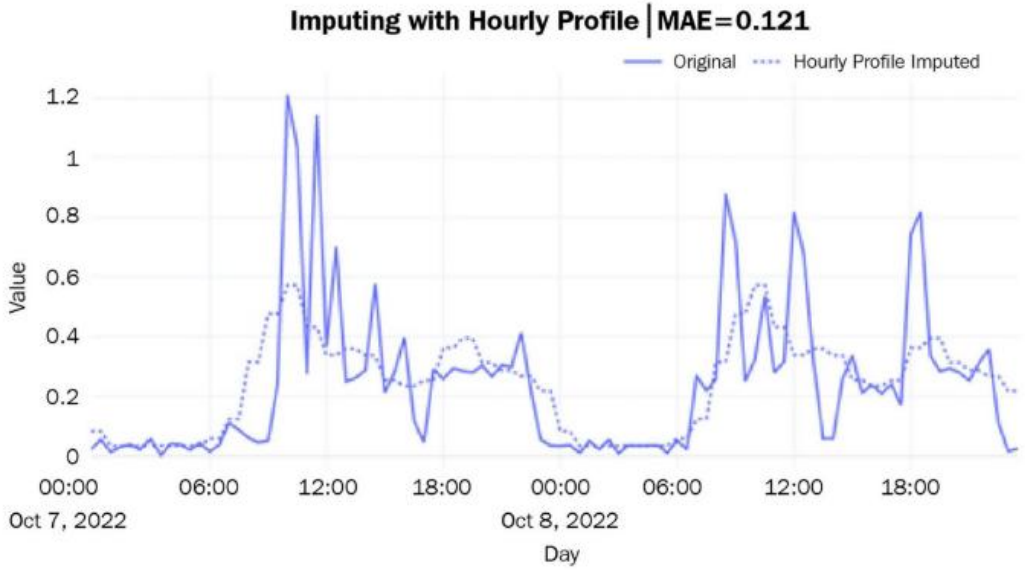


Figure 2.13: Imputing with an hourly profile

The hourly average for each weekday

```
#Create a column with the weekday from timestamp
ts_df["weekday"] = ts_df.index.weekday
#Calculate weekday-hourly average consumption
day_hourly_profile = ts_df.groupby(['weekday','hour'])['energy_consumption'].
mean().reset_index()
day_hourly_profile.rename(columns={"energy_consumption": "day_hourly_profile"}
inplace=True)
#Saving the index because it gets lost in merge
```

```
idx = ts_df.index
#Merge the day-hourly profile dataframe to ts dataframe
ts_df = ts_df.merge(day_hourly_profile, on=['weekday', 'hour'], how='left',
validate="many_to_one")
ts_df.index = idx
#Using the day-hourly profile to fill missing
ts_df['day_hourly_profile_imputed'] = ts_df['energy_consumption_missing']
ts_df.loc[null_mask,"day_hourly_profile_imputed"] = ts_df.loc[null_mask,"day_
hourly_profile"]
mae = mean_absolute_error(ts_df.loc[window, "day_hourly_profile_imputed"], ts_
df.loc[window, "energy_consumption"])
```

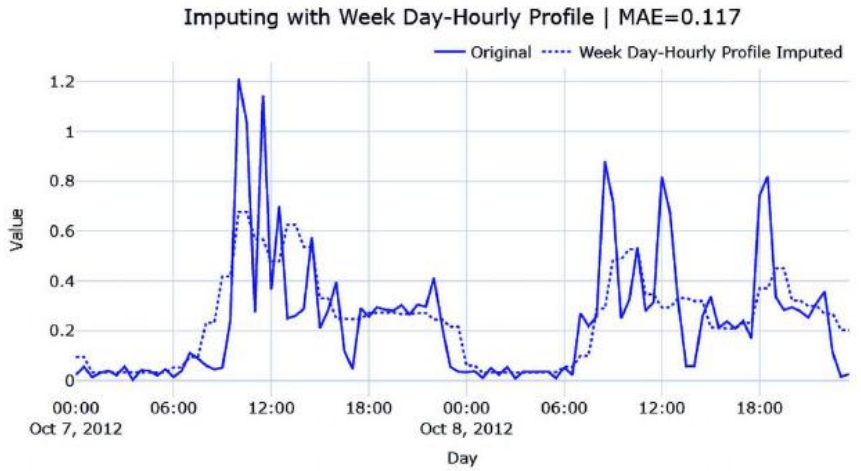


Figure 2.14: Imputing the hourly average for each weekday

Seasonal interpolation

Although calculating seasonal profiles and using them to impute works well, there are instances, especially when there is a trend in the time series, where such a simple technique falls short.

The simple seasonal profile doesn't **capture** the **trend** at all and **ignores** it completely. For such cases, we can do the following:

1. Calculate the seasonal profile, similar to how we calculated the averages earlier.

2. Subtract the seasonal profile and apply any of the interpolation techniques we saw earlier.

3. Return the seasonal profile to the interpolated series.

Seasonal interpolation

```
from src.imputation.interpolation import SeasonalInterpolation
# Seasonal interpolation using 48*7 as the seasonal period.
recovered_matrix_seas_interp_weekday_half_hour =
SeasonalInterpolation(seasonal_period=48*7,decomposition_strategy="additive",
interpolation_strategy="spline", interpolation_args={"order":3}, min_value=0).
fit_transform(ts_df.energy_consumption_missing.values.reshape(-1,1))
ts_df['seas_interp_weekday_half_hour_imputed'] = recovered_matrix_seas_interp_
weekday_half_hour
```

Here, we have done seasonal interpolation using 48 (half-hourly) and 48*7 (weekday to half-hourly) and plotted the resulting imputation:

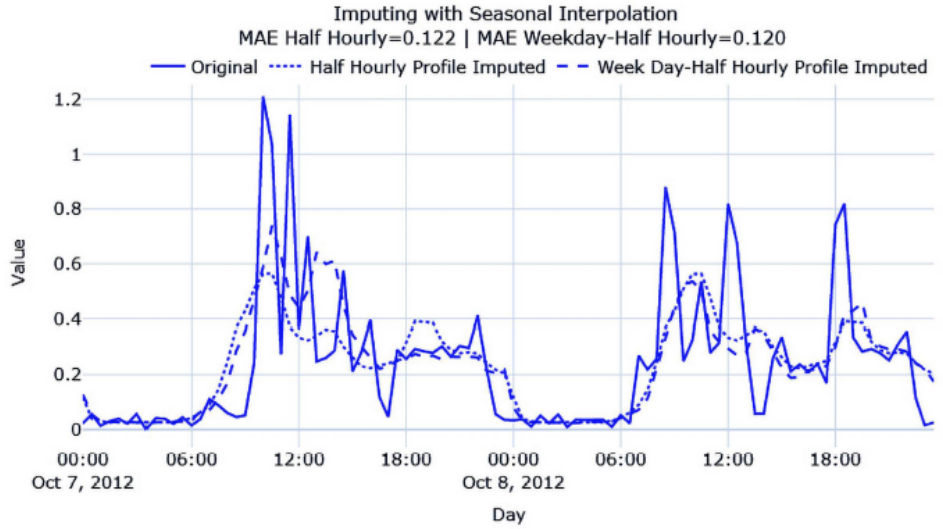


Figure 2.15: Imputing with seasonal interpolation

Feature Engineering

Feature engineering **creates new features** that help the model learn better patterns.

The goal is to create features that are **predictive**, **informative**, and **not unnecessarily redundant**.

Hand-Crafted vs Automated Features

Hand-crafted features are designed **manually** based on **domain knowledge**.

Automated feature extraction uses **libraries** or algorithms to **generate** many **features automatically**.

Examples include **Catch22**, **HCTSA**, **tsfresh**, **featuretools**, **ROCKET**, and **shapelets**.

Interpretable Features

Interpretable features are easy to understand, such as

mean, **maximum**, **minimum**, **trend**, or **rolling statistics**.

They are useful when **explainability** is important.

Types of Time-Series Features

Important feature groups include:

- Date-related features
- Time-related features
- Calendar features
- Window-based features
- Convolutional features
- Shapelet features

Causality and Leakage

Time-series **features** must only use **past** and **present** information.

Using **future** data creates **leakage** and gives overly optimistic results.

A valid preprocessing pipeline must respect the prediction time.

Date- and Time-Related Features

Date and time variables contain information about **dates**, **time**, or a combination (datetime).

If we want to feed these fields into a machine learning model, we need to derive **relevant information**.

We can **significantly improve** the **performance** of our machine learning model by **enriching** our **dataset** with these **extracted features**.

Holiday Features

Holidays can **affect behavior** in many domains, such as sales, traffic, finance, or web activity.

Libraries such as workalendar can generate country-specific or region-specific holiday calendars.

```
from workalendar.europe.united_kingdom import UnitedKingdom
UnitedKingdom().holidays()
```

```
[(datetime.date(2021, 1, 1), 'New year'),
 (datetime.date(2021, 4, 2), 'Good Friday'),
 (datetime.date(2021, 4, 4), 'Easter Sunday'),
 (datetime.date(2021, 4, 5), 'Easter Monday'),
 (datetime.date(2021, 5, 3), 'Early May Bank Holiday'),
 (datetime.date(2021, 5, 31), 'Spring Bank Holiday'),
 (datetime.date(2021, 8, 30), 'Late Summer Bank Holiday'),
 (datetime.date(2021, 12, 25), 'Christmas Day'),
 (datetime.date(2021, 12, 26), 'Boxing Day'),
 (datetime.date(2021, 12, 27), 'Christmas Shift'),
 (datetime.date(2021, 12, 28), 'Boxing Day Shift')]
```

Holiday Features

Similarly, we can get holidays for other places, for example, California, USA. We can also extract lists of holidays, and add custom holidays:

```
from typing import List
from dateutil.relativedelta import relativedelta, TH
import datetime
from workalendar.usa import California
def create_custom_holidays(year: int) -> List:
      custom_holidays = California().holidays()
      custom_holidays.append((
        (datetime.datetime(year, 11, 1) + relativedelta(weekday=TH(+4))
+ datetime.timedelta(days=1)).date(),
        "Black Friday"
      ))
      return {k: v for (k, v) in custom_holidays}

custom_holidays = create_custom_holidays(2021)
```

```
{datetime.date(2021, 1, 1): 'New year',
 datetime.date(2021, 1, 18): 'Birthday of Martin Luther King, Jr.',
 datetime.date(2021, 2, 15): "Washington's Birthday",
 datetime.date(2021, 3, 31): 'Cesar Chavez Day',
 datetime.date(2021, 5, 31): 'Memorial Day',
 datetime.date(2021, 7, 4): 'Independence Day',
 datetime.date(2021, 7, 5): 'Independence Day (Observed)',
 datetime.date(2021, 9, 6): 'Labor Day',
 datetime.date(2021, 11, 11): 'Veterans Day',
 datetime.date(2021, 11, 25): 'Thanksgiving Day',
 datetime.date(2021, 11, 26): 'Thanksgiving Friday',
 datetime.date(2021, 12, 24): 'Christmas Day (Observed)',
 datetime.date(2021, 12, 25): 'Christmas Day',
 datetime.date(2021, 12, 31): 'New Years Day (Observed)',
 datetime.date(2016, 11, 25): 'Black Friday'}
```

Holiday Features

```
def create_custom_holidays(year: int) -> List:
```

```
def is_holiday(current_date: datetime.date):
    """Determine if we have a holiday."""
    return custom_holidays.get(current_date, False)

today = datetime.date(2021, 4, 11)
is_holiday(today)
```

This can be a very **useful feature** for a machine learning model. For example, we could imagine a **different profile** of users who **apply** for loans on bank **holidays** or on a **weekday**.

Date Annotation

Date annotation extracts information such as:

- Day of week
- Number of days since start of year
- Number of days until end of year
- Days since start of month
- Days until end of month

Date Annotation

```
import calendar
calendar.monthrange(2021, 1)
```

```
from datetime import date
def year_anchor(current_date: datetime.date):
      return (
        (current_date - date(current_date.year, 1, 1)).days,
        (date(current_date.year, 12, 31) - current_date).days,
      )

year_anchor(today)
```

```
def month_anchor(current_date: datetime.date):
      last_day = calendar.monthrange(current_date.year, current_date.
month)[0]

      return (
        (current_date - datetime.date(current_date.year, current_date.
month, 1)).days,
        (current_date - datetime.date(current_date.year, current_date.
month, last_day)).days,
      )

month_anchor(today)
```

Payday Features

We could imagine that some people get paid in the middle or at the end of the month, and would then access our website to buy our products. Most people would get paid on the last Friday of the month.

```
def get_last_friday(current_date: datetime.date, weekday=calendar.
FRIDAY):
      return max(week[weekday]
        for week in calendar.monthcalendar(
            current_date.year, current_date.month
        ))

get_last_friday(today)
```

Season Features

Seasons can capture cyclic behavior across the year.
Examples: winter, spring, summer, and autumn.
Seasonal features can improve models when behavior changes across the year.

```
YEAR = 2021
seasons = [
    ('winter', (date(YEAR,  1,  1),  date(YEAR,  3, 20))),
    ('spring', (date(YEAR,  3, 21),  date(YEAR,  6, 20))),
    ('summer', (date(YEAR,  6, 21),  date(YEAR,  9, 22))),
    ('autumn', (date(YEAR,  9, 23),  date(YEAR, 12, 20))),
    ('winter', (date(YEAR, 12, 21),  date(YEAR, 12, 31)))
]

def is_in_interval(current_date: datetime.date, seasons):
      return next(season for season, (start, end) in seasons
                  if start <= current_date.replace(year=YEAR) <= end)

is_in_interval(today, seasons)
```

Sun and Moon Features

Astronomical features such as daylight hours, sunrise, dusk, or moon phase may be useful in some domains.
For example, more daylight may correlate with more business activity or mobility.

```
from astral.sun import sun
from astral import LocationInfo
CITY = LocationInfo("London", "England", "Europe/London", 51.5, -0.116)
def get_sunrise_dusk(current_date: datetime.date, city_name='London'):
      s = sun(CITY.observer, date=current_date)
      sunrise = s['sunrise']
      dusk = s['dusk']
      return (sunrise - dusk).seconds / 3600

get_sunrise_dusk(today)
```

We are getting `9.788055555555555` hours of daylight.

Business Days

The number of business days and non-business days in a month can affect monthly activity.This is useful for sales, operations, finance, and demand forecasting.

```
def get_business_days(current_date: datetime.date):
      last_day = calendar.monthrange(current_date.year, current_date.
month)[1]
      rng = pd.date_range(current_date.replace(day=1), periods=last_
day, freq='D')
      business_days = pd.bdate_range(rng[0], rng[-1])
      return len(business_days), last_day - len(business_days)

get_business_days(date.today())
```

We should be getting `(22, 9)` – 22 business days and 9 weekend days and holidays.

Automated Feature Extraction

Libraries such as **featuretools** and **tsfresh** can automatically generate many time-based features.
Featuretools can **extract features** such as minute, hour, day, month, year, and weekday.
tsfresh can generate a large number of statistical descriptors.

```
import featuretools as ft
from featuretools.primitives import Minute, Hour, Day, Month, Year,
Weekday

data = pd.DataFrame(
    {'Time': ['2014-01-01 01:41:50',
              '2014-01-01 02:06:50',
              '2014-01-01 02:31:50',
              '2014-01-01 02:56:50',
              '2014-01-01 03:21:50'],
     'Target': [0, 0, 0, 0, 1]}
)
data['index'] = data.index
es = ft.EntitySet('My EntitySet')
es.entity_from_dataframe(
    entity_id='main_data_table',
    index='index',
    dataframe=data,
    time_index='Time'
)
fm, features = ft.dfs(
    entityset=es,
    target_entity='main_data_table',
    trans_primitives=[Minute, Hour, Day, Month, Year, Weekday]
)
```

Automated Feature Extraction

Our features are `Minute`, `Hour`, `Day`, `Month`, `Year`, and `Weekday`. Here's our DataFrame, `fm`:

index	Target	DAY(Time)	HOUR(Time)	MINUTE(Time)	MONTH(Time)	WEEKDAY(Time)	YEAR(Time)
0	0	1	1	41	1	2	2014
1	0	1	2	6	1	2	2014
2	0	1	2	31	1	2	2014
3	0	1	2	56	1	2	2014
4	1	1	3	21	1	2	2014

Figure 3.6: Featuretools output

ROCKET

The research paper *ROCKET: Exceptionally fast and accurate time-series classification using random convolutional kernels* (Angus Dempster, François Petitjean, Geoffrey I. Webb; 2019) presents a novel methodology for convolving time-series data with **random kernels** that can result in higher accuracy and faster training times for machine learning models. What makes this paper unique is banking on the recent successes of convolutional neural networks and transferring them to preprocessing for time-series datasets.

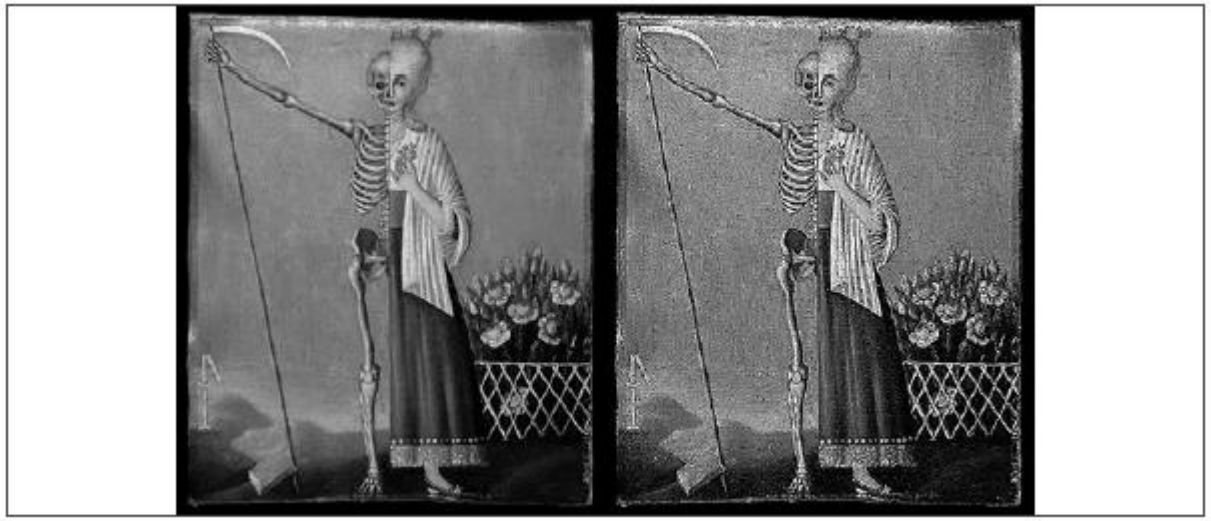


Figure 3.1: Sharpening filter

Convolutions

A convolution applies a **kernel** over local parts of the **data**.

In images, kernels can sharpen or detect edges.

In time-series, **kernels** detect **local** temporal **patterns**.

ROCKET

We can improve machine learning accuracy from time-series by applying transformations with convolutional kernels.

Each feature gets transformed by **random kernels**, the number of which is a parameter to the algorithm. This is set to 10,000 by default.

The transformed features can now be fed as input into any machine learning algorithm. The authors propose to use **linear algorithms** like ridge regression classifier or logistic regression.

ROCKET

The idea of ROCKET is very similar to Convolutional Neural Networks (CNNs), two big differences are:

1. ROCKET doesn't use any hidden layers or non-linearities

2. The convolutions are applied independently for each feature

Converts a Time Series into Features

ROCKET applies many random convolutional kernels to a time series.
Each kernel slides over the series and produces a new sequence of responses.

This response sequence shows how strongly the kernel matches local patterns in different parts of the time series.

Instead of keeping the full response sequence, ROCKET summarizes it using two scalar features:

Max value — the strongest response produced by the kernel.
PPV — Positive Proportion Value — the proportion of responses that are positive.

Therefore, one kernel transforms an entire time series into only two numbers;
i.e., 1000 kernels create 2000 features (columns) for each time series.

Max and PPV Features

Max captures the **strongest** occurrence of a **pattern**.
It answers the question:
“How strongly did this kernel’s pattern appear anywhere in the time series?”

PPV captures how **frequently** the **pattern** appears.
It answers the question:
“In how many parts of the time series did this kernel produce a positive response?”

For example, if a convolution produces:
[-0.5, 4.5, 0, 1.5]

Then:
Max = 4.5
PPV = 2 / 4 = 0.5

ROCKET uses thousands of random kernels, so the final output is a large feature vector that summarizes many different temporal patterns.

Shapelets

Shapelets are discriminative subsequences of a time-series.

They represent **local patterns** that help distinguish between classes.

A model can classify a time-series by comparing it to learned shapelets.

How Shapelets Work?

Imagine you are trying to classify **two** types of medical **heartbeats** (ECG signals): **Healthy** vs. **Unhealthy**.

A global alignment method (like Dynamic Time Warping) would compare the entire length of the two signals.

A **shapelet** approach searches for a **small**, **specific dip** or **spike** in the waveform that ***only*** appears in the unhealthy signals.

Once the algorithm identifies this specific subsequence (the shapelet), it classifies new time series based on a simple rule: **"How close is the nearest match of this shapelet within the new time series?"**

Advantages of Shapelets

Shapelets can provide **interpretable** results.

They can be **fast** during **prediction** once the shapelet dictionary is learned.

They often achieve **competitive performance** in time-series classification.

It is **independent** of the **time** axis (Shift Invariant). The model looks for the **shape**, and it doesn't matter to it whether it appeared at the **beginning**, **middle**, or **end** of the series.

Python Practice: ROCKET

```
from sktime.datasets import load_arrow_head
from sktime.utils.data_processing import from_nested_to_2d_array
X_train, y_train = load_arrow_head(split="train", return_X_y=True)
from_nested_to_2d_array(X_train).head()
```

We get an unnested DataFrame like this:

	dim_0__0	dim_0__1	dim_0__2	dim_0__3	dim_0__4	dim_0__5	dim_0__6	dim_0__7	dim_0__8	dim_0__9	...	d
0	-1.9630	-1.9578	-1.9561	-1.9383	-1.8967	-1.8699	-1.8387	-1.8123	-1.7364	-1.6733	...	
1	-1.7746	-1.7740	-1.7766	-1.7307	-1.6963	-1.6574	-1.6362	-1.6098	-1.5434	-1.4862	...	
2	-1.8660	-1.8420	-1.8350	-1.8119	-1.7644	-1.7077	-1.6483	-1.5826	-1.5315	-1.4936	...	
3	-2.0738	-2.0733	-2.0446	-2.0383	-1.9590	-1.8745	-1.8056	-1.7310	-1.7127	-1.6280	...	
4	-1.7463	-1.7413	-1.7227	-1.6986	-1.6772	-1.6304	-1.5794	-1.5512	-1.4740	-1.4594	...	

5 rows × 251 columns

Figure 3.7: ROCKET features

```
from sktime.transformations.panel.rocket import Rocket
rocket = Rocket(num_kernels=1000)
rocket.fit(X_train)
X_train_transform = rocket.transform(X_train)
```

Python Practice: Shapelets

```
from sktime.transformations.panel.shapelets import
ContractedShapeletTransform
shapelets_transform = ContractedShapeletTransform(
    time_contract_in_mins=1,
    num_candidates_to_sample_per_case=10,
    verbose=0,
)
shapelets_transform.fit(X_train, y_train)
```

```
X_train_transform = shapelets_transform.transform(X_train)
```

Key Takeaways

Preprocessing is essential for reliable time-series machine learning.

Feature transforms improve data distributions.

Imputation handles missing values.

Feature engineering extracts useful temporal information.

ROCKET and shapelets provide advanced time-series representations.

Summary

The goal of preprocessing is to convert raw time-series data into reliable, informative, and model-ready features.

Good preprocessing improves accuracy, reduces noise, avoids leakage, and makes the learning problem easier.